

Lab Objectives :

The objective of this Lab exercise is to demonstrate FSM design for widely practical applications like AXI4 Lite protocol.

Question No. 01

Design an FPGA module that implements an AXI4-Lite interface to communicate with on chip BRAM. The design will receive commands and data from the testbench, perform read/write on the BRAM and display data when required. Make a Verilog top module named AXI4_LITE_TOP, within which you need to instantiate two separate Verilog modules, AXI4_LITE_MASTER and AXI4_LITE_SLAVE, each containing one FSM as per the AXI4 Lite protocol.

The AXI4_LITE_MASTER module should support two commands:

CMD_WRITE (1'b1): Write data into the BRAM

CMD_READ (1'b0): Read data from the BRAM

The AXI4_LITE_TOP module should have the following I/O ports:

Inputs:

CLK: Input Clock

RESETN: Input Active Low Reset

START: Input signal to indicate start of operation

CMD_IN: 16-bit input command (details below)

DATA_IN: 32-bit input data to be written into the BRAM

Outputs:

DATA_OUT: 32-bit output data being read from the BRAM

GET_DATA: output signal which when asserted, absorbs DATA_IN from testbench

GET_CMD: output signal which when asserted, absorbs CMD_IN from testbench

The testbench provided should be used directly without any modifications, so it is important to follow the above module declaration.

The 16b input command (CMD_IN) communication should be configured as follows:

- MSB 1-bit: Read/Write (1'b1 for CMD_WRITE; 1'b0 for CMD_READ)
- Next 7-bits: Size of data (number of words to read/write, range: 1 – 128)
- Last 8-bits: Initial memory address (0x00 – 0xFF) from where to start reading/writing

Decode these 2 bytes of CMD_IN, and if it is a write command, the GET_DATA signal needs to be asserted. Then you need to generate required control signals for the AXI4-Lite interface. The AXI4_LITE_MASTER module should contain an FSM which generates AXI4 Lite signals (corresponding to five channels - AW, W, B, AR and R) based on the input command. These signals would be read by the AXI4_LITE_SLAVE module to generate control signals for the BRAM and the AXI4_LITE_MASTER module itself.

Since the initial memory address is represented using 8 bits, the maximum possible BRAM size is 256 locations. Configure the BRAM with a word width of 32 bits and a depth of 256, using a single-port RAM, with a “native” interface type. Instantiate this BRAM inside the AXI4_LITE_SLAVE module, within which you need to write the Verilog code corresponding to an AXI4 Lite Slave FSM. Assume the 4-bit AXI_WSTRB signal will be either 0xF or 0x0, to

enable and disable writes into the BRAM respectively (no byte-wise write enables).

Target Device is Xilinx Artix-7 XC7A35T- ICPG236C (Family Artix-7, Part XC7A35T, Package CPG236, Speed Grade -1). Do the functional simulation. After elaboration, verify the RTL schematic. Do the implementation, timing analysis, and post-route timing simulation.

Report the area (resource utilization) and performance (f_{max}).

Submit the source Verilog codes, post-route timing simulation waveforms, and a brief report on the resource utilization in terms of Slices/LUTs and delay performance.

Architecture :

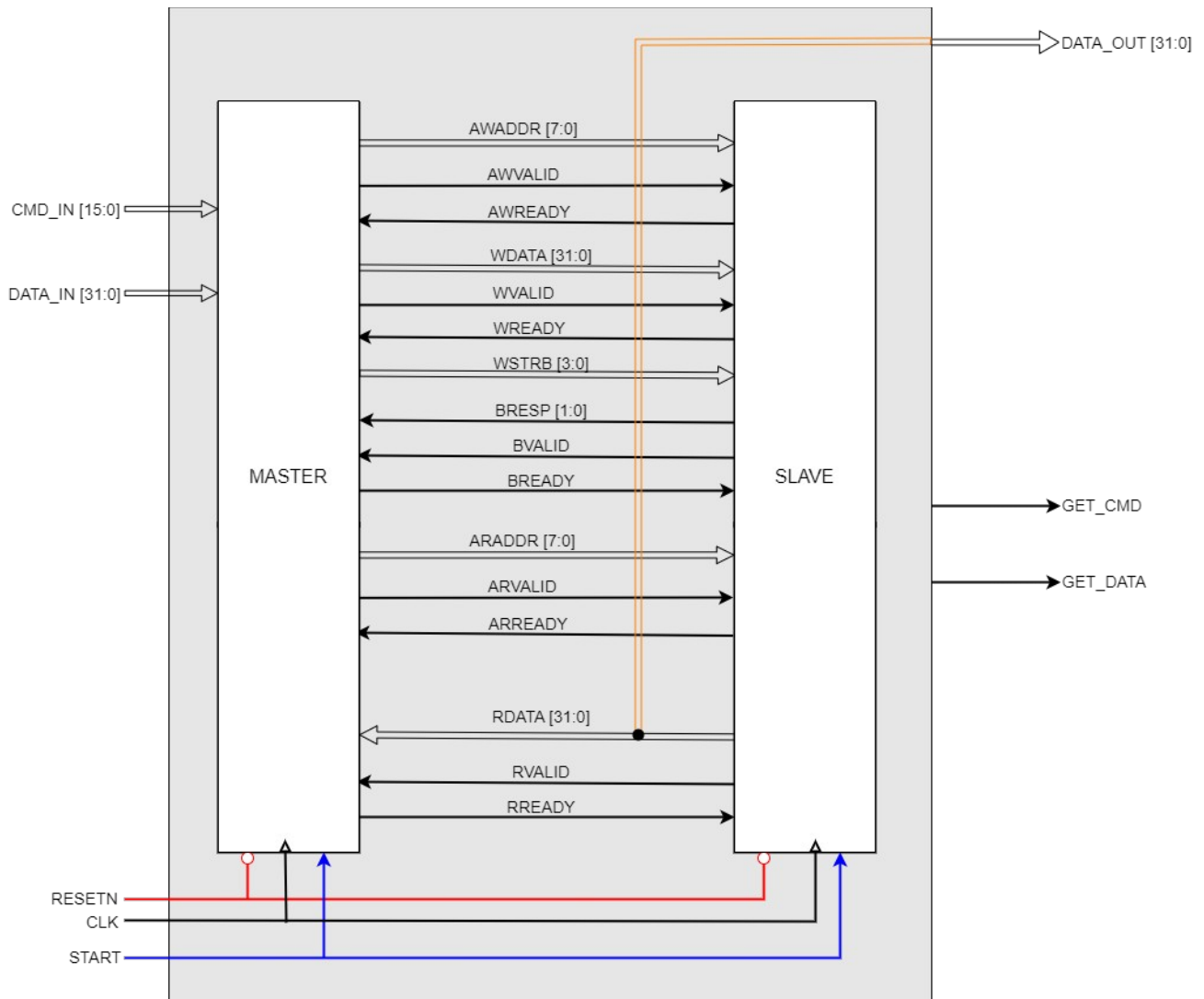


Figure 01: The TOP module architecture for AXI4 Lite Protocol Handshake between MASTER and SLAVE module

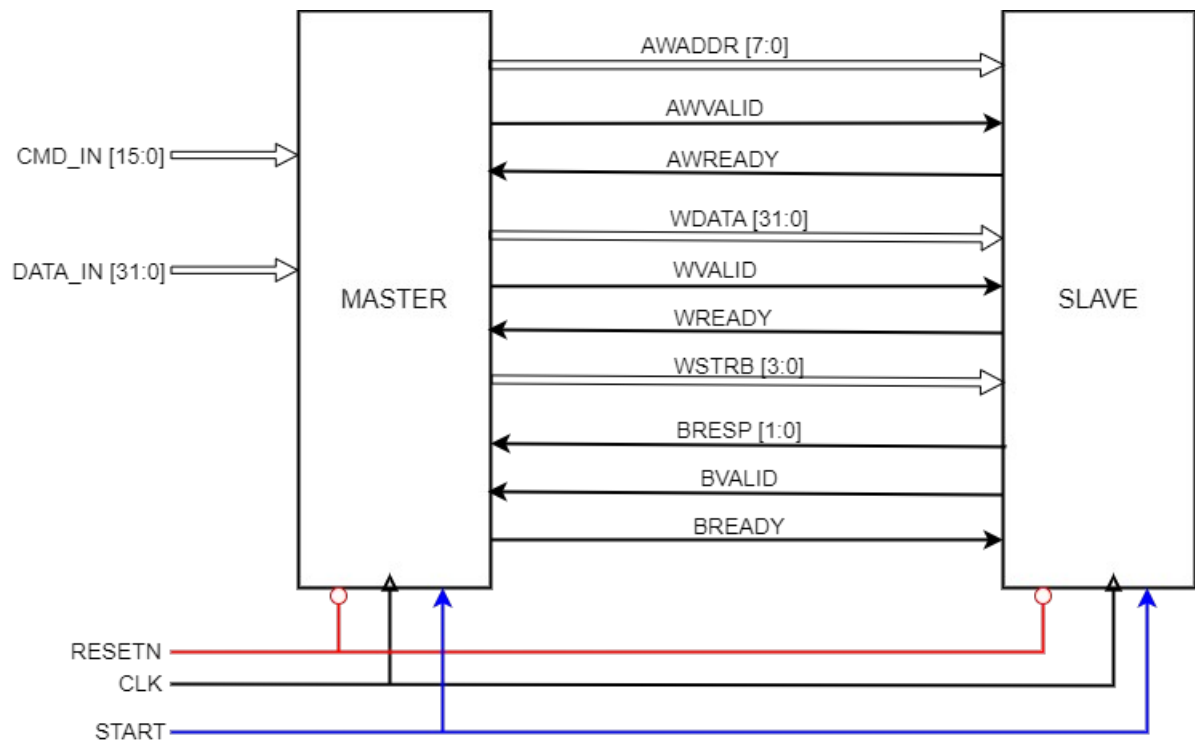


Figure 02: The signal flow of Write operation for AXI4 Lite protocol between MASTER and SLAVE module

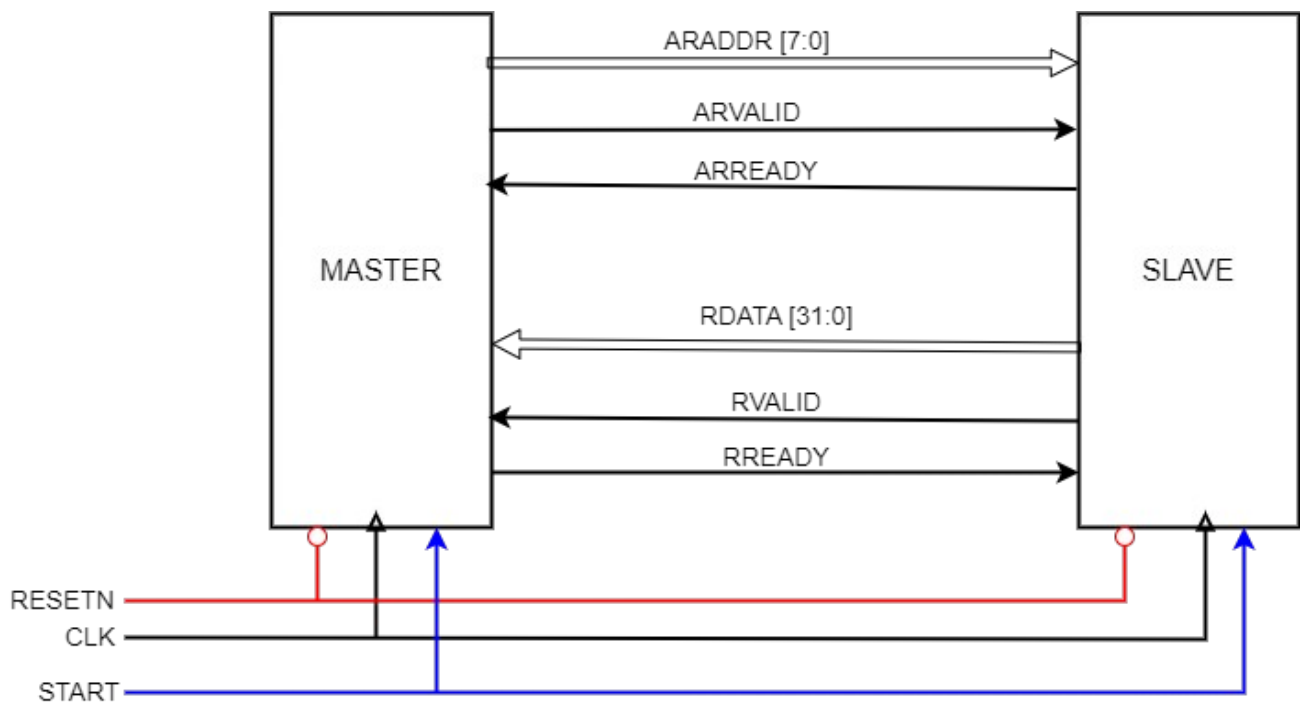


Figure 03: The signal flow of Read operation for AXI4 Lite protocol between MASTER and SLAVE module

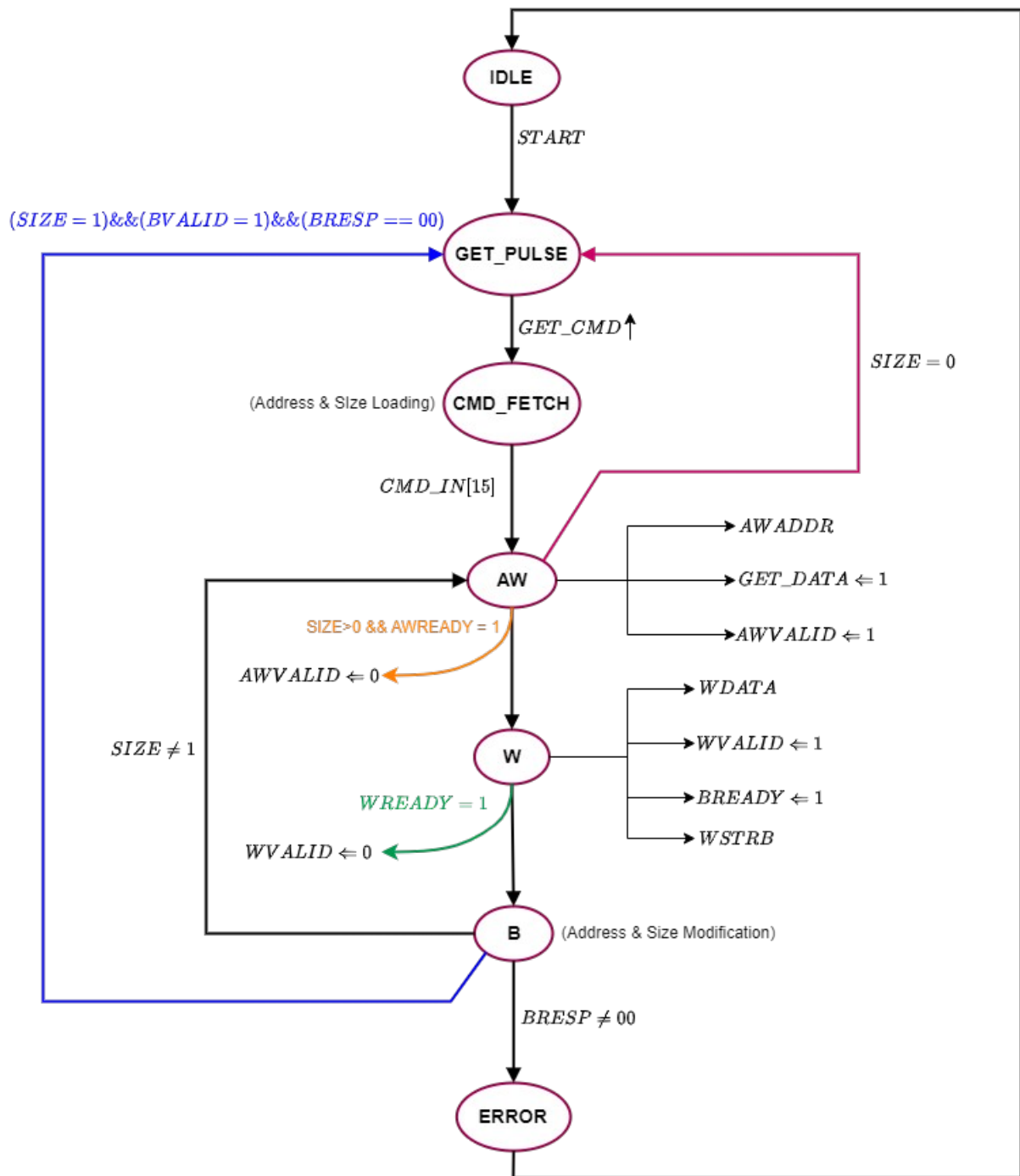


Figure 04 : The FSM architecture or design for Write operation of MASTER module

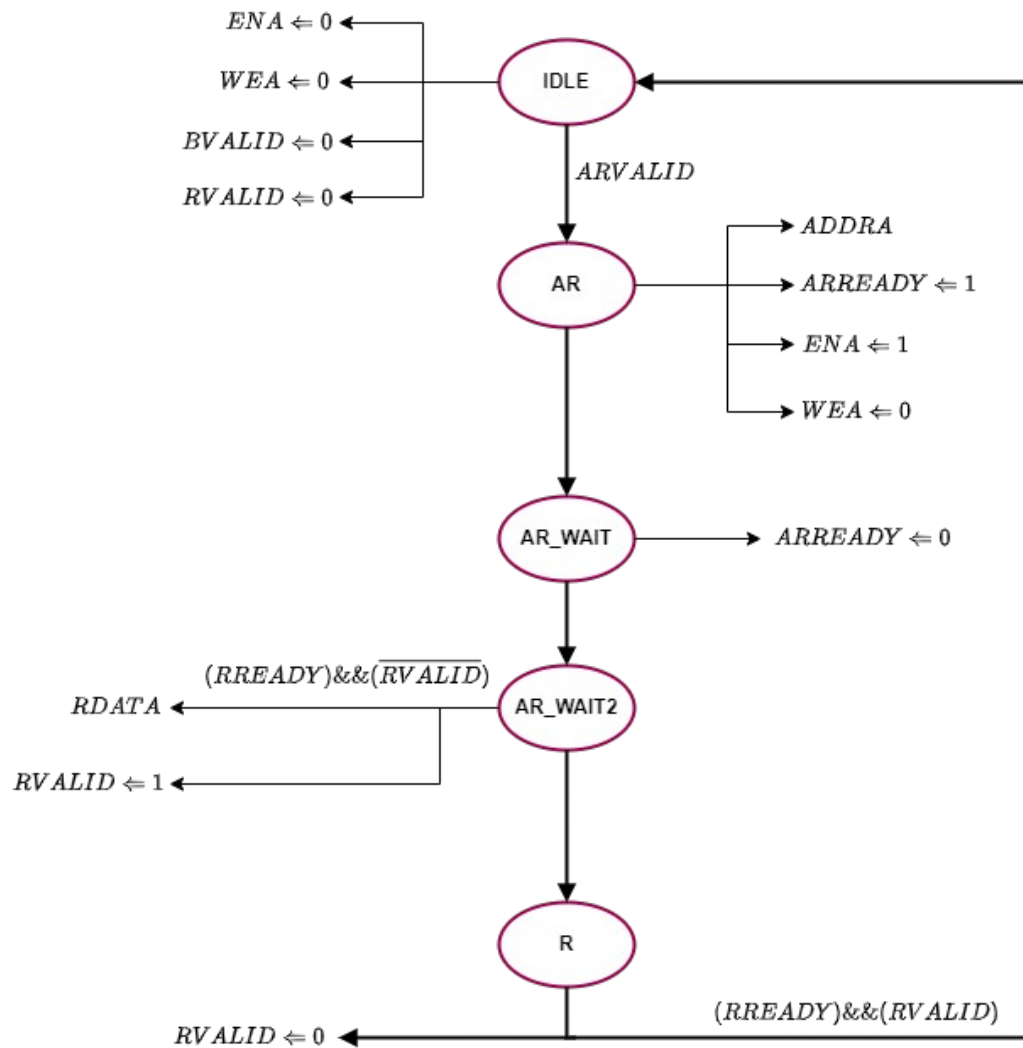


Figure 07: The FSM architecture or design for Read operation of SLAVE module

Verilog Code :

```

`timescale 1ns / 1ps
/////////////////////////////////////////////////////////////////
// Company:
// Engineer: Amit Barman
//
// Create Date: 04.03.2026 09:32:09
// Design Name: AXI4 Lite Handshake
// Module Name: AXI4_Handshake
// Project Name: Assignment 3 AXI4 Handshake
// Target Devices: Artix-7 XC7A35T-ICPG236C
// Tool Versions:
// Description:
//
// Dependencies:
//
  
```

```
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
////////////////////////////////////
```

```
module AXI4_LITE_TOP(
    input CLK, RESETN, START,
    input [15:0] CMD_IN,
    input [31:0] DATA_IN,
    output wire [31:0] DATA_OUT,
    output wire GET_DATA, GET_CMD
);
```

```
    // AXI signals
    wire [7:0] AWADDR, ARADDR;
    wire [31:0] WDATA, RDATA;
    wire AWREADY, WREADY, ARREADY, BVALID, RVALID, AWVALID, WVALID,
    ARVALID, BREADY, RREADY;
    wire [1:0] BRESP;
    wire [3:0] WSTRB;
```

```
    assign DATA_OUT = RDATA;
```

```
    AXI4_LITE_MASTER u1(
        .clk(CLK),
        .resetn(RESETN),
        .start(START),
        .CMD_IN(CMD_IN),
        .DATA_IN(DATA_IN),
        .GET_CMD(GET_CMD),
        .GET_DATA(GET_DATA),
        .AWREADY(AWREADY),
        .WREADY(WREADY),
        .ARREADY(ARREADY),
        .BVALID(BVALID),
        .RVALID(RVALID),
        .BRESP(BRESP),
        .AWADDR(AWADDR),
        .ARADDR(ARADDR),
        .WDATA(WDATA),
        .WSTRB(WSTRB),
        .AWVALID(AWVALID),
        .WVALID(WVALID),
        .ARVALID(ARVALID),
        .BREADY(BREADY),
        .RREADY(RREADY)
    );
```

```
    AXI4_LITE_SLAVE u2(
```

```
.clk(CLK),
.resetn(RESETN),
.AWVALID(AWVALID),
.WVALID(WVALID),
.ARVALID(ARVALID),
.AWADDR(AWADDR),
.ARADDR(ARADDR),
.WDATA(WDATA),
.WSTRB(WSTRB),
.BREADY(BREADY),
.RREADY(RREADY),
.BRESP(BRESP),
.RDATA(RDATA),
.AWREADY(AWREADY),
.WREADY(WREADY),
.ARREADY(ARREADY),
.BVALID(BVALID),
.RVALID(RVALID)
);
```

```
endmodule
```

```
module AXI4_LITE_MASTER(
    input clk, resetn, start,
    input [15:0] CMD_IN,
    input [31:0] DATA_IN,
    input AWREADY, WREADY, ARREADY, BVALID, RVALID,
    input [1:0] BRESP,

    output reg [7:0] AWADDR, ARADDR,
    output reg [31:0] WDATA, //RDATA,
    output reg [3:0] WSTRB,
    output reg AWVALID, WVALID, ARVALID,
    output reg BREADY, RREADY,
    output reg GET_DATA, GET_CMD
);

    reg [3:0] state;
    reg [7:0] current_address;
    reg [6:0] size_count;

    localparam IDLE = 4'd0;
    localparam GET_PULSE = 4'd1;
    localparam CMD_FETCH = 4'd2;
    localparam AW = 4'd3;
    localparam W = 4'd4;
    localparam B = 4'd5;
    localparam AR = 4'd6;
    localparam R = 4'd7;
    localparam DONE = 4'd8;
    localparam ERROR = 4'd9;
```

```

always@(posedge clk) begin
    if(!resetn) begin
        state <= IDLE;
        AWVALID <= 0;
        WVALID <= 0;
        BREADY <= 0;
        ARVALID <= 0;
        RREADY <= 0;
        GET_CMD <= 0;
        GET_DATA <= 0;
        AWADDR <= 0;
        ARADDR <= 0;
        WDATA <= 0;
        WSTRB <= 0;
        current_address <= 0;
        size_count <= 0;
    end
    else begin
        GET_CMD <= 0;
        GET_DATA <= 0;
        case(state)
            IDLE: begin
                if (start)
                    state <= GET_PULSE;
            end

            GET_PULSE: begin
                GET_CMD <= 1;
                state <= CMD_FETCH;
            end

            CMD_FETCH: begin
                current_address <= CMD_IN[7:0];
                size_count <= CMD_IN[14:8];
                if (CMD_IN[15] == 1) begin
                    state <= AW;
                end
                else begin
                    state <= AR;
                end
            end

            AW: begin
                if (size_count > 0) begin
                    GET_DATA <= 1;
                    AWVALID <= 1;
                    AWADDR <= current_address;
                    if (AWREADY) begin
                        state <= W;
                        AWVALID <= 0;
                    end
                end
            end
        endcase
    end
end

```

```

        end
        else begin
//          GET_CMD <= 1;
            state <= GET_PULSE;
        end

        end

W: begin
    WDATA <= DATA_IN;
    WVALID <= 1;
    BREADY <= 1;
   WSTRB <= 4'b1111;
    if (WREADY) begin
        state <= B;
        WVALID <= 0;
    end
end

B: begin
//    BREADY <= 1;
//    if (BVALID && (~AWVALID || AWREADY) && (~WVALID || WREADY))
begin
    if (BVALID) begin
        BREADY <= 0;
        if (BRESP == 2'b00) begin //okay signal
            if (size_count == 7'd1) begin
                current_address <= current_address;
            end
            else begin
                current_address <= current_address + 1'b1;
            end
            size_count <= size_count - 1'b1;
            if (size_count == 1)
                state <= GET_PULSE;
            else begin
                state <= AW;
            end
        end
    end
    else begin
        state <= ERROR;
    end
end
end

AR: begin
    if (size_count > 0) begin
        ARADDR <= current_address;
        ARVALID <= 1;
    end
end

```

```

        RREADY <= 1;
        if (ARREADY) begin
            state <= R;
            ARVALID <= 0;
        end
    end
else begin
//      GET_CMD <= 1;
        state <= DONE;
    end
end

R: begin
//      RREADY <= 0;
    if (RVALID && RREADY) begin
        RREADY <= 0;
        if (size_count == 7'd1) begin
            current_address <= current_address;
        end
        else begin
            current_address <= current_address + 1;
        end
        size_count <= size_count - 1;
        if (size_count == 1) begin
            state <= DONE; //DONE
        end
        else
            state <= AR;
        end
    end
end

DONE: state <= GET_PULSE;
ERROR: state <= IDLE;
default : state <= IDLE;
endcase
end

end
endmodule

module AXI4_LITE_SLAVE(
    input clk, resetn,
    input AWVALID, WVALID, ARVALID,
    input [7:0] AWADDR, ARADDR,
    input [31:0] WDATA,
    input [3:0] WSTRB,
    input BREADY, RREADY,
    output reg [1:0] BRESP,
    output reg [31:0] RDATA,
    output reg AWREADY, WREADY, ARREADY, BVALID, RVALID

```

```

);

reg ENA;
reg [0:0] WEA;
reg [7:0] ADDRA;
reg [31:0] DINA;
wire [31:0] DOUTA;

BRAM_0 bram_2_clkL (
    .clka(clk),    // input wire clka
    .ena(ENA),     // input wire ena
    .wea(WEA),     // input wire [0 : 0] wea
    .addra(ADDRA), // input wire [7 : 0] addra
    .dina(DINA),   // input wire [31 : 0] dina
    .douta(DOUTA)  // output wire [31 : 0] douta
);

reg [3:0] state;

localparam IDLE = 4'd0;
localparam AW = 4'd1;
localparam W = 4'd2;
localparam B = 4'd3;
localparam AR = 4'd4;
localparam AR_WAIT = 4'd5;
localparam AR_WAIT2 = 4'd6;
localparam R = 4'd7;
localparam DONE = 4'd8;

always@(posedge clk) begin
    if (!resetn) begin
        state <= IDLE;
        AWREADY <= 0;
        WREADY <= 0;
        ARREADY <= 0;
        BVALID <= 0;
        RVALID <= 0;
        BRESP <= 2'b00;
        RDATA <= 0;

        ENA <= 0;
        WEA <= 0;
        ADDRA <= 0;
        DINA <= 0;
    end
    else begin
        AWREADY <= 0;
        WREADY <= 0;
        ARREADY <= 0;
    end
end

```

```

case(state)
  IDLE: begin
    ENA <= 0;
    WEA <= 0;
    BVALID <= 0;
    RVALID <= 0;
    if (AWVALID)
      state <= AW;
    else if (ARVALID)
      state <= AR;
    else
      state <= IDLE;
    end

  AW: begin
    // RDATA <= 0;
    AWREADY <= 1;
    ADDRA <= AWADDR;
    state <= W;
    end

  W: begin
    if (WVALID) begin
      WREADY <= 1;
      state <= B;
      ENA <= 1;

      if (WSTRB == 4'b1111) begin
        DINA <= WDATA;
        WEA <= 1;
        BRESP <= 2'b00;    //okay
      end
      else begin
        WEA <= 0;
        BRESP <= 2'b10;    //slave error
      end
    end
  end

  B: begin
    AWREADY <= 0;
    WREADY <= 0;
    ENA <= 0;
    WEA <= 0;
    if ((~BVALID) || BREADY) BVALID <= 1;
    if (BREADY && BVALID) begin
      state <= IDLE; //DONE
      BVALID <= 0;
    end
  end
end

```

```

end

AR: begin
    state <= AR_WAIT; //AR_WAIT
    ARREADY <= 1;
    ENA <= 1;
    WEA <= 0;
    ADDRA <= ARADDR;
//    RDATA_temp <= DOUTA;
end

AR_WAIT:begin
//    RVALID <= 1;
    state <= AR_WAIT2;
    ARREADY <= 0;
end

AR_WAIT2:begin
    state <= R;
end

R: begin
    if (RREADY || (~RVALID)) begin
        RVALID <= 1;
        RDATA <= DOUTA;
    end
    if (RREADY && RVALID) begin
        RVALID <= 0;
        state <= IDLE; //DONE
    end
end

DONE: state <= IDLE;
default: state <= IDLE;
endcase
end

end
endmodule

```

RTL Block Diagram :

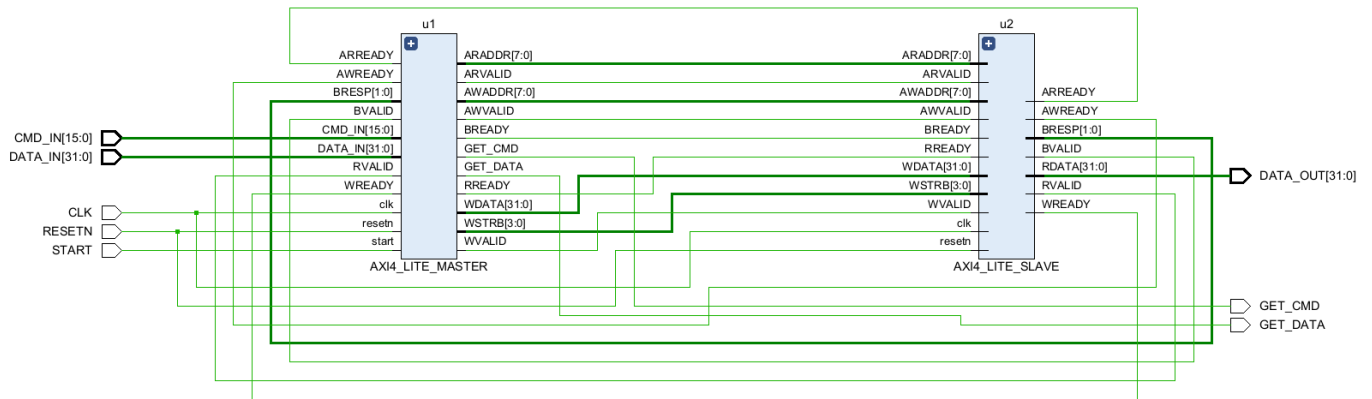


Figure 08: RTL Block diagram of AXI4 Lite Handshaking protocol

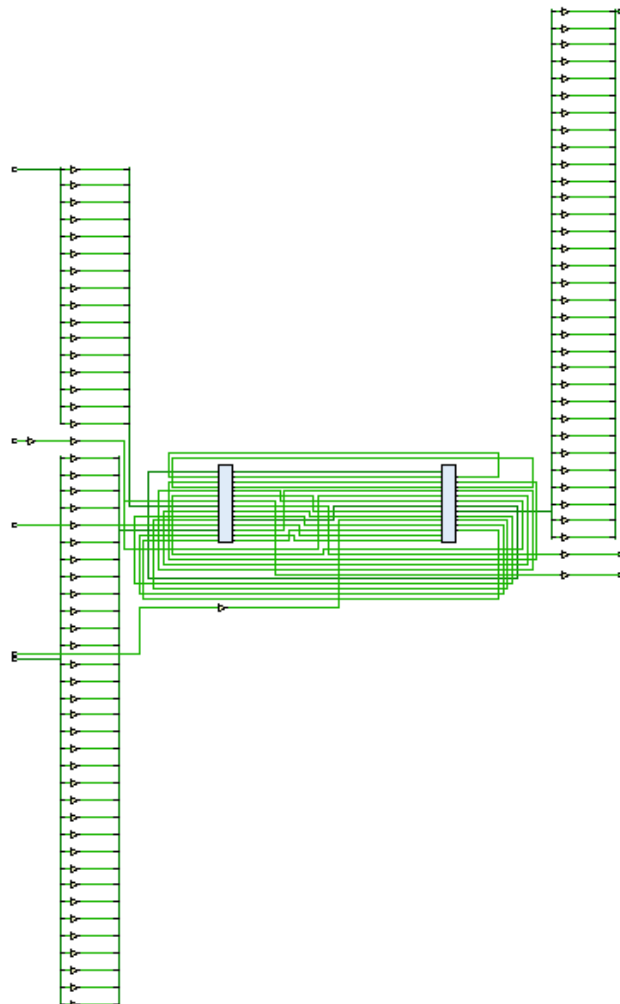


Figure 09 : Schematic circuit diagram of AXI4 Lite protocol performing module

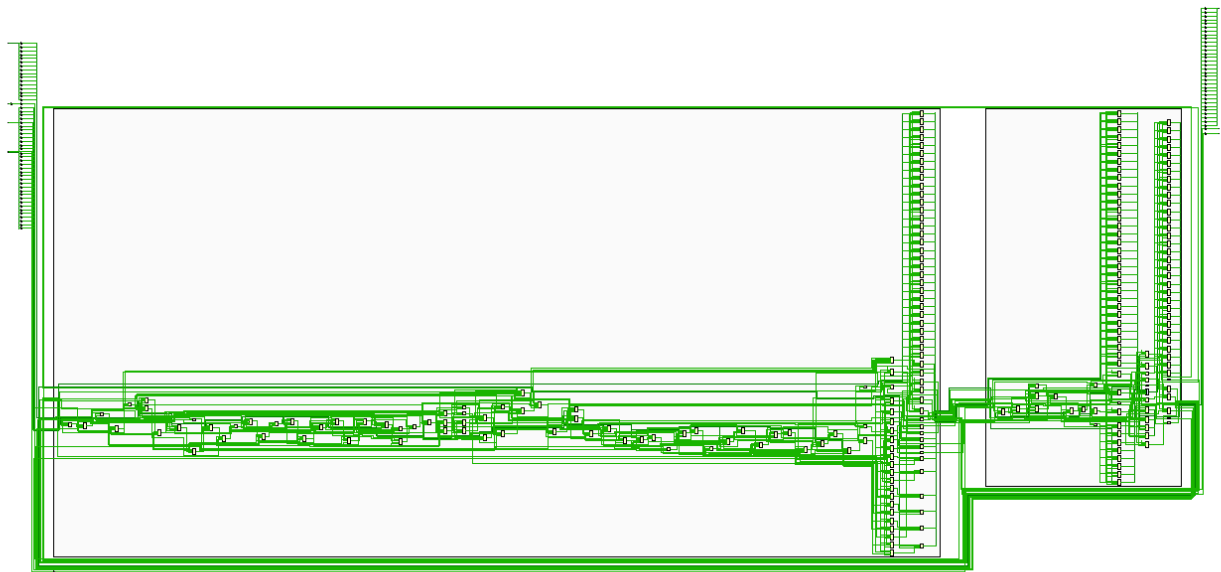


Figure 10: Schematic Diagram of AXI4 Lite protocol performing module (Expanded View)

Resource Utilization :

Summary

Resource	Utilization	Available	Utilization %
LUT	62	20800	0.30
FF	169	41600	0.41
BRAM	0.50	50	1.00
IO	85	106	80.19

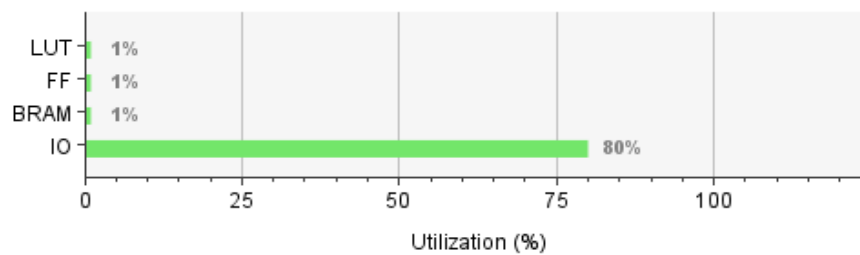


Figure 11: Summary of resource utilization for the AXI4 Lite handshake performing module

Power Analysis :

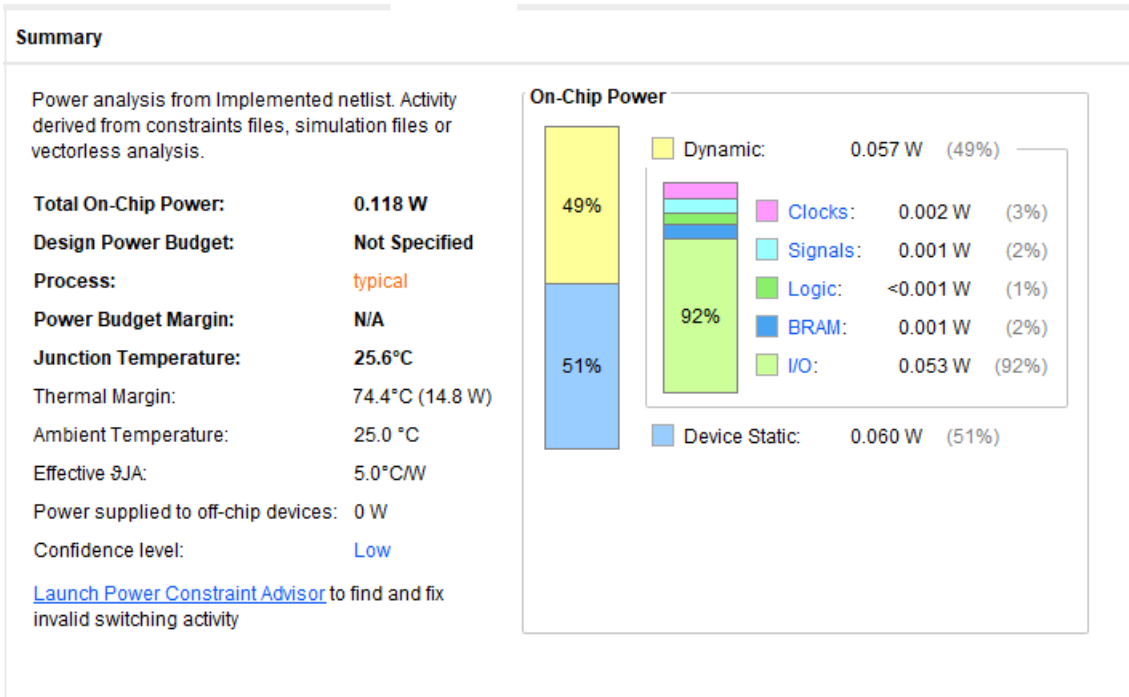


Figure 12 : Summary of Power analysis for AXI4 Lite handshake performing module

Timing Report :

Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 0.523 ns	Worst Hold Slack (WHS): 0.081 ns	Worst Pulse Width Slack (WPWS): 4.500 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 583	Total Number of Endpoints: 583	Total Number of Endpoints: 172

All user specified timing constraints are met.

Figure 13 : Summary of timing report of AXI4 Lite handshake performing module

The maximum theoretical clock frequency that can be achieved for this AXI4 Lite protocol performing module to operate correctly,

$$f_{max} = \frac{1}{(\text{Applied CLK period} - \text{Worst Setup slack})}$$

$$f_{max} = \frac{1}{(6 - 0.523) \times 10^{-9}} = 182.582 \text{ MHz}$$

By introducing multiple pipelining, we can increase the maximum frequency more above from this 87.13MHz.

Waveforms :

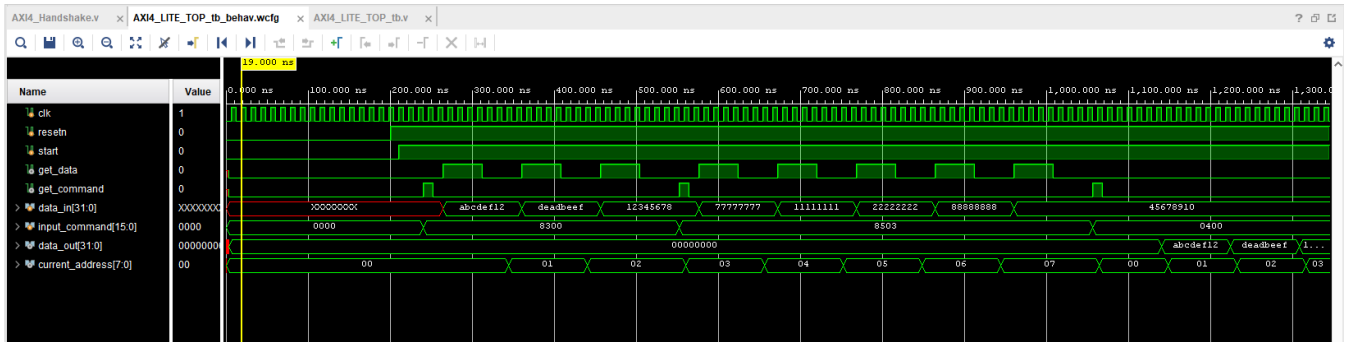


Figure 14 : Post Synthesis timing waveform (1) of the AXI4 Lite protocol performing module

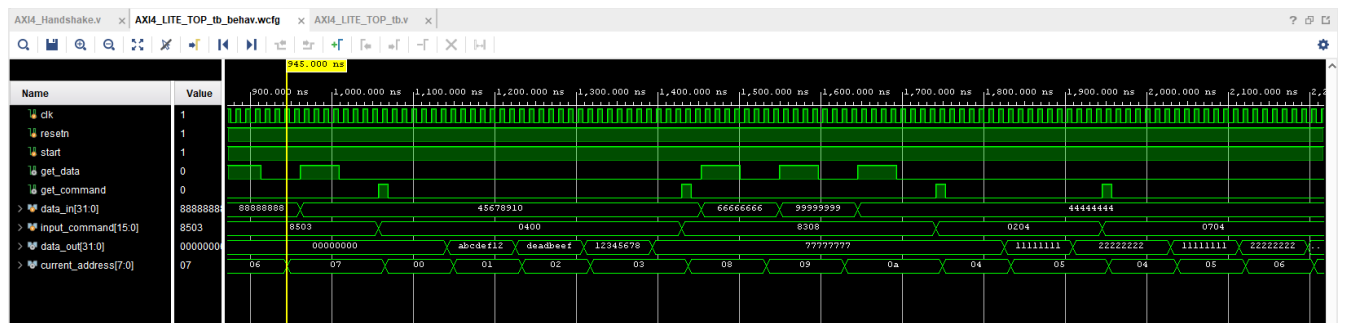


Figure 15 : Post Synthesis timing waveform (2) of the AXI4 Lite protocol performing module

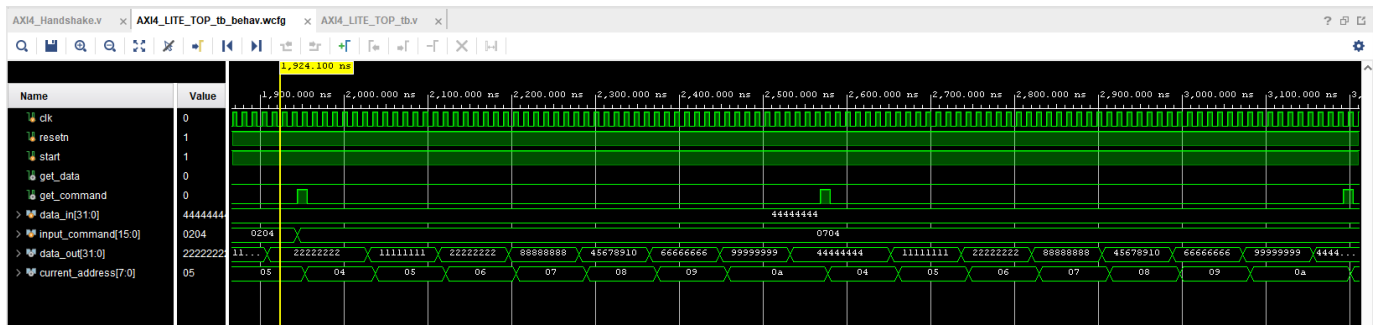


Figure 16 : Post Synthesis timing waveform (3) of the AXI4 Lite protocol performing module

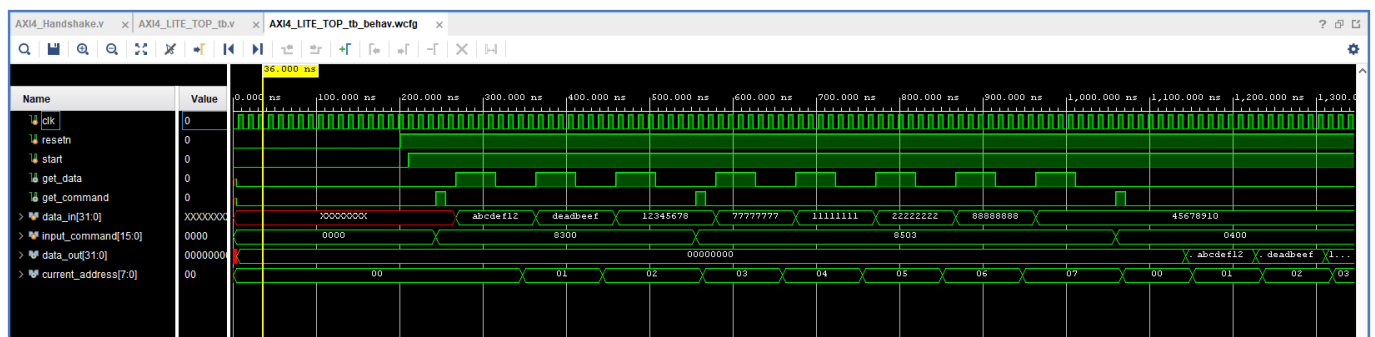


Figure 17 : Post Implementation timing waveform (1) of the AXI4 Lite protocol performing module

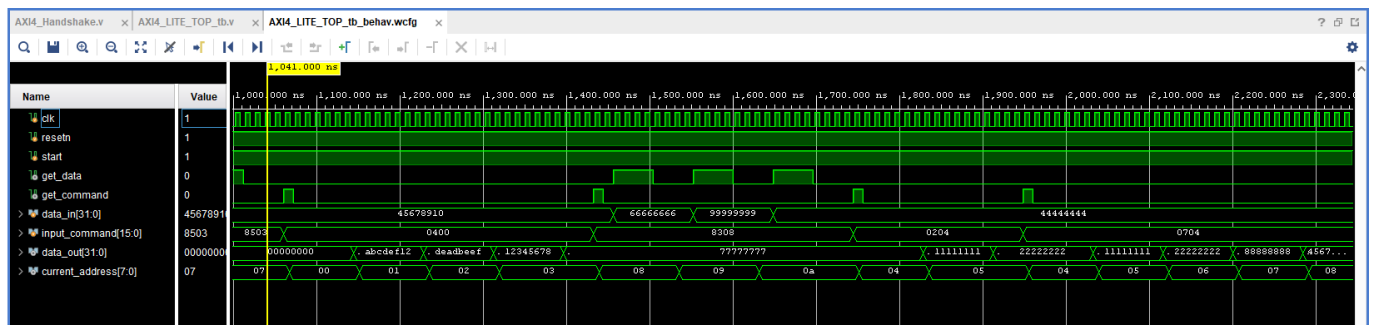


Figure 18 : Post Implementation timing waveform (2) of the AXI4 Lite protocol performing module

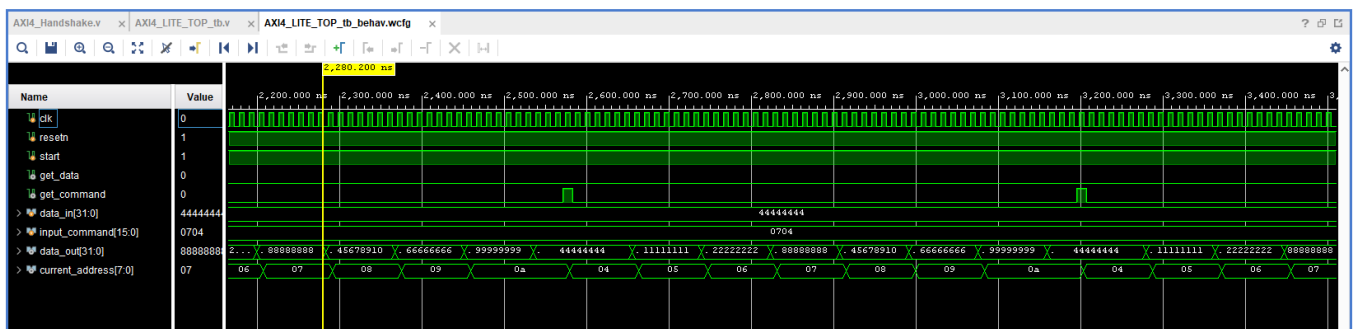


Figure 19 : Post Implementation timing waveform (3) of the AXI4 Lite protocol performing module